

[Approach and Solution Outline]

For every generator and discriminator of both DCGAN and CycleGAN, I didn't use MaxPool or Upsample layers which are not learnable because they can discard important spatial information, making training instable. I only used learnable strided convolution (Conv or ConvTranspose layers) with 'stride=2' for both downsampling and upsampling. Also, I set batch_size for DCGAN as 128 to reduce training time and stable training, and I used all pre-defined hyperparameters for CycleGAN. To struct specific architectures of the networks, I asked AI tool (Gemini) for advice on what great structures are and built them. At first, I structured the generators of CycleGAN with 4 residual blocks, but the performance of the model seemed not that good by visual inspection, especially in generating SVHN images. So, I added 2 more residual blocks to mitigate the issues by enhancing capacity of the model.

DCGAN Architectures

Discriminator: It takes images as input and downsapmls them through 4 Conv layers. The output is a set of probability of whether the inputs are real or fake images generated by the generator.

Generator: It takes 100-dimensional latent noise vector as input and upsamples it through 4 ConvTranspose layers. In the last layer, instead of BatchNorm and ReLU, Tanh is used to limit pixel values in $[-1, 1]$ range.

CycleGAN Architecture

Discriminators (Dx & Dy): They are similar to that of DCGAN, but it has 3 Conv blocks and it classifies input into one of the 10 digit classes (0~9) or 'fake' class.

Generators (netG & netF): They consist of an encoder, 6 residual blocks, and decoder. Both encoder and decoder have 3 Conv and ConvTranspose layers. The residual blocks are located in the bottleneck of the architecture. They allow higher performance by building a deeper network.

[Results and Analysis] – Answers for Analysis Questions**1. GAN vs. CycleGAN****- Architecture**

DCGAN: It consists of one generator and one discriminator. The generator takes 100-dimensional latent noise vector as input to generate a new real-looking image. Then, the discriminator distinguishes whether the image is real or fake.

CycleGAN: It has more complex architecture with two generators and two discriminators. Each generator takes an image from one domain as input and generates an image in the style of the other domain. Each discriminator distinguishes whether the image is real one from its domain or fake one transferred by the opposing generator.

- Training Complexity

DCGAN: The training process is relatively simple. It optimizes a single adversarial loss, where the generator tries to fool the discriminator, and the discriminator tries to distinguish real and fake more strictly.

CycleGAN: The training process is more complex. In addition to two adversarial losses for each generator-discriminator pair, it introduces a new concept, cycle consistency loss. This ensures that the reconstructed image processed by two generators sequentially is similar to the original image. Then, the total objective of training is a weighted sum of those three losses, requiring more computation and careful balancing.

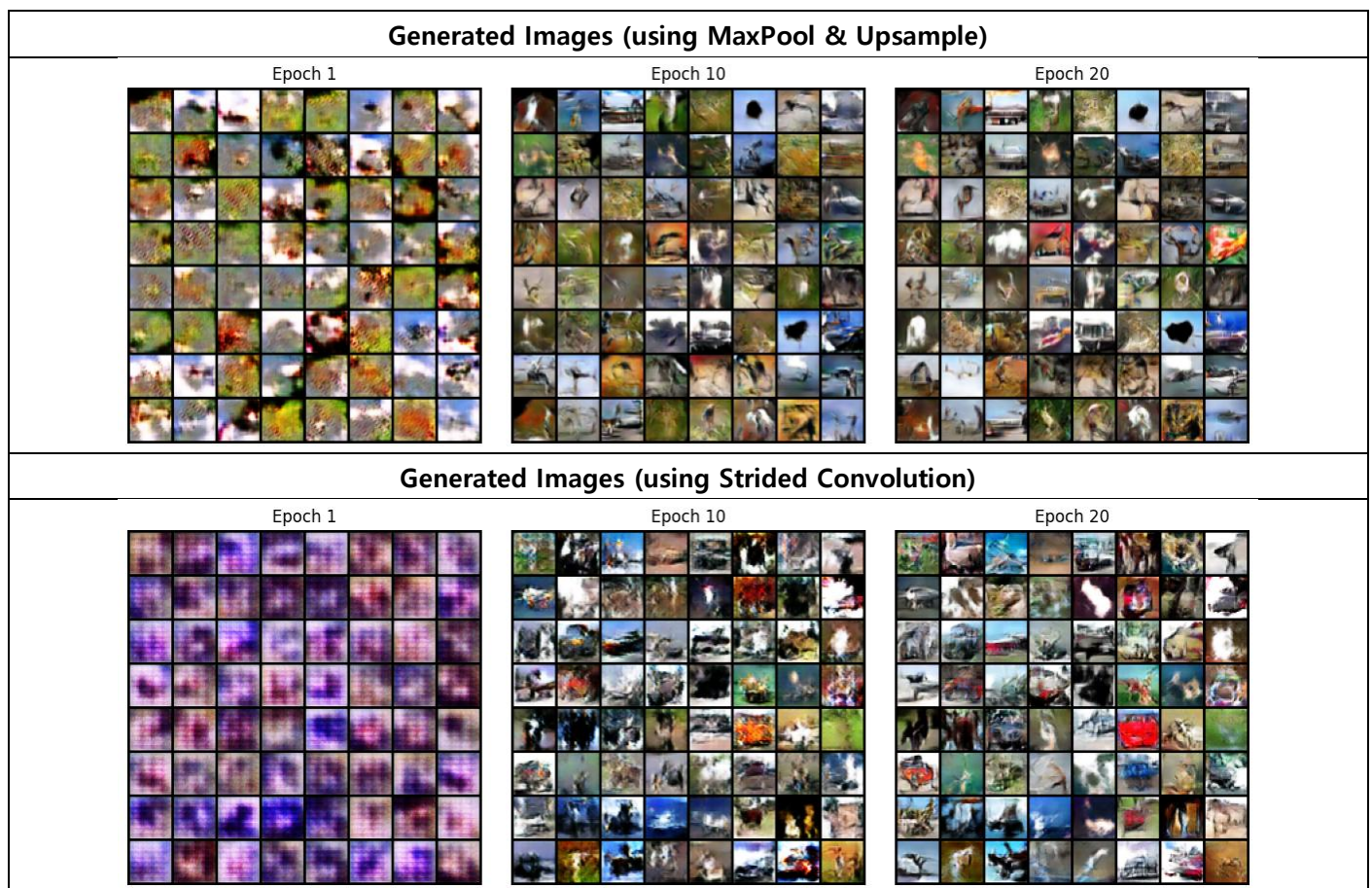
- Different Roles of Generators and Discriminators

DCGAN: The generator acts as a creator. It creates a new image from random noise. Then, the discriminator acts as an examiner. It judges whether an image is real or fake.

CycleGAN: The generators act as style-transferer. They convert an image from one style to another, preserving its content. Then, the discriminators act as style-critics. They determines whether an image has the style of a certain domain.

2. Challenges Faced

DCGAN: It had challenges of training instability and mode collapse. Initially, I constructed both generator and discriminator using MaxPool and Upsample layer for downsampling and upsampling like in traditional CNN architecture. However, the 'Loss_G' decreased to 2.32 and then grew again over 2.87 as training progressed, indicating training instability. FID score was also high, 60.03. And it showed slight mode collapse like figures below. According to them, the images generated by the generator using MaxPool and Upsample layers are more similar to each other than those using strided convolution. They have more similar backgrounds (grass or sky), shapes, and colors. To address these issues, I changed the structures of the networks. I only used strided convolution (Conv and ConvTranspose) layers for downsampling and upsampling. Then, the 'Loss_G' gradually decreased to 1.96, and FID score also decreased to 57.77.



CycleGAN: The main challenge with CycleGAN was achieving high-quality generated images, especially in SVHN style. Initially, I constructed the generators with 4 residual blocks. However, according to the figures below, the style-transferred images from generators lacked sufficient detail and clarity and were significantly blurred. So, I decided to add 2 more residual blocks in the middle of the architecture of the generators to mitigate the issue. Then, by deepening the network and increasing capacity, the visual quality of the generated images slightly improved, that is, the proportion of images which are crushed or blurred slightly decreases.

MNIST → SVHN (4 Residual blocks)

MNIST → SVHN (6 Residual blocks)

SVHN → MNIST (4 Residual blocks)	SVHN → MNIST (6 Residual blocks)

3. Quantitative Metrics

When comparing the initial DCGAN model using MaxPool and Upsample layers with improved model using strided convolution, FID score showed correlation with improvement in visual quality. The improved model with lower FID score also showed more stable training process and generated more diverse images than that with higher FID score. In this case, FID score is useful quantitative metric in evaluating generative models. However, the change of FID score didn't fully reflect all issues which can be assessed through visual evaluation. While FID score dropped only about 2.26 points (from 60.03 to 57.77), the diversity of generated images was much improved; the backgrounds, shapes, and colors are very different from each other.

Thus, in cases where the quantitative metric and qualitative visual assessment disagreed, I trusted the visual assessment more, because the goal of generative models is creating diverse images (or other types of data) that are convincing to human, which is not calculated into mathematical digits.

The quantitative metrics like FID score have values of objectivity and automation. Unlike subjective visual evaluation, FID score provides a single objective score, making it easier to track the degree of improvement of performance automatically. However, it has limitations in capturing diversity of data distribution. In my experiment, it didn't fully reflect the improvement of mode collapse. Also, another limitation it has is bias toward pre-trained dataset. Since FID score computed based on the output of pre-trained ImageNet, the reliability of it would decrease for dataset with different distributions than the pre-trained one.

4. Applications and Ethics

- Real-world application

DCGAN: [Data augmentation in medical imaging] For training models to diagnose rare diseases, it needs sufficient data, but those data are sensitive and lack. Then, by augmenting some medical images using DCGAN, we can secure a large amount of real-looking data without invasion of privacy of patients.

CycleGAN: [Simulation-to-Real domain adaptation in robotics] In robotics, training models like robotic arms or self-driving cars are difficult, dangerous, and expensive. Using CycleGAN, it can transfer real-world images into realistic style images, which can be used in simulation for training those models.

- Ethical considerations or potential misuse: Deepfakes or fake images

The main problems or potential misuse of generative models are creating malicious contents of fake images (or deepfakes). Generative models can generate images or videos where someone says or does something he or she

never did. For example, the fake video of a politician saying fake speech might spread out.

- Responsibilities ML practitioners have when developing or deploying

ML practitioners have responsibilities to research and develop detection techniques to identify deep fakes or other misuses. Also, they must label clear marks on AI-generated contents to prevent them being misused, and apply strict terms of services to prohibit malicious use.

5. Conceptual Understanding

If the discriminator classifies only two classes (real or fake), then the generator will learn to ignore the content of the input, generating similar plausible output regardless of input. For example, if the discriminator classifies a certain '1' image as 'real', then the generator tends to keep generating the '1' images to fool the discriminator. Then, the adversarial loss becomes lower since the generator successfully fools the discriminator, but it shows severe mode collapse generating only '1' images regardless of input. While the adversarial loss pushes the generator to produce '1', the cycle consistency loss enforces the generator to generate the original image. This conflict leads training very unstable, and the model fails to learn meaningful style transfer for all classes.

6. Results Discussion

Evaluation Section 1.5

- Visual Inspection: I displayed a grid of generated images in part **[2. Challenges Faced]**. At epoch 1, they look still just noises, but they become relatively realistic compared to those at epoch 1 as training progresses. They become less crushed and blurred. Also, they exhibit diversity. After replacing MaxPool and Upsample layers into strided convolution, the generated images look more different from each other in terms of backgrounds, shapes, or colors.

- FID: I compute FID score based on 5000 real and 5000 fake samples. The score is 57.77.

- Discussion: The followings are examples of mode collapse for both models using MaxPool & Upsample and using strided convolution. We can say these are failure modes. However, there are a lot of diverse generated samples in the grids.



Evaluation Section 2.5

- Visual Quality: Two figures with generated images below are constructed for cycle consistency checks by visual assessment. According to them, the results seem to hold consistency somewhat well. Also, they look like real numbers in both MNIST and SVHN styles.

- Cycle Consistency Check:

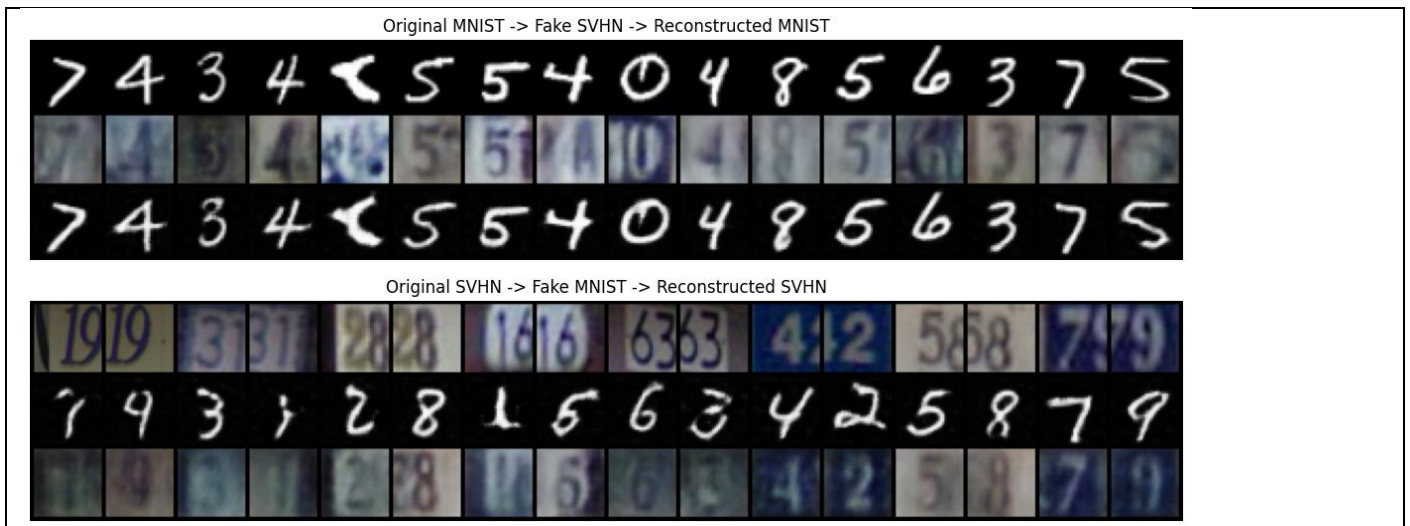
Cycle Consistency L1 Loss (MNIST): 0.0817

Cycle Consistency L1 Loss (SVHN): 0.1512

Cycle Consistency L2 Loss (MNIST): 0.0359

Cycle Consistency L2 Loss (SVHN): 0.0401

- Training Stability: As I mentioned in part **[2. Challenges Faced]**, even after training 1000 iterations, the losses tend to converge but the visual quality of generated images, especially in SVHN style, were not good. So, I changed the internal architecture of generator networks by adding 2 more Residual blocks in the bottleneck. Then, the losses converged and the model reconstructed images more well.



[Use of AI]

(a) Which tools or platforms have you used?

I have mainly used Gemini, and sometimes accepted codes suggested by autocompletion of Colab.

(b) For what tasks did you use AI and how?

- Code implementation: I asked for AI to get code snippets of functions in both part 1 and part 2 such as MyLSTMCell, MyLSTM, MyLSTM_att, Generator, Discriminator, Residual_Block, Train_D_ms, etc.
- Debugging: In the skeleton code of part 2, there was an issue with NameError or AttributeError because the letter 'F' was used for both 'torch.nn.functional as F' and the generator 'F' of CycleGAN. After encountering those errors, I asked for AI to know which parts induce the errors, and modified them.
- Understanding concepts: I used AI to see the exact equation of FID score and the meaning of it.

(c) How you integrated the AI's suggestions into your work.

For proposed codes, I made an effort to understand the meaning of each line. If there are some lines which I can't understand, then I asked for AI about the role of those lines. For example, in implementing GAN, AI suggested me some codes with '.detach()'. At first, I didn't understand why it use the method, so I asked AI "what happens if I don't write '.detach()' method?" before utilizing it.

(d) Any instances where AI was wrong or not directly helpful.

In implementing GAN, it recommended code using nn.MaxPool2d for downsampling, but it might not be suitable for higher performance since it is not learnable. So, I asked another question to AI that whether there is another way to upsampling and downsampling features. Then, I replaced those layers into strided convolution layers.

(e) Citation of AI content.

Among those parts applied code snippets generated by AI, especially the parts for implementing MyLSTMCell and MyLSTM_att in part 1 and for implementing Generator, training loop, computing FID score, Residual_Block, defining genOptimizer and disOptimizer using itertools module, visualizing $X \rightarrow Y \rightarrow X$ or $Y \rightarrow X \rightarrow Y$ translations and computing cycle consistency checks, were significantly generated by AI. I also marked them in ipynb files.